

Case Study

Rover communication system

Contents

1. Prerequisite	3
2. Main point to think about	4
2.1. How to identify where to send to	4
2.2. Resolver tags	4
2.3. Problem	4
2.4. Solution	4
3. How do you define the structure of the message	4
3.1. How to pack different data (Just examples)	5
3.1.1. For example, Motor Drive Command (Command ID: 0x12)	5
3.1.2. Another example, receiving IMU Sensor Readings (Command ID: 0x35) .	5
4. How do you handle receiving and processing packets at high speed?	5

1. Prerequisite

This case study is for the Robo-team interview with Ethan Bastian.

The assignment asked us to design a **general communication system** that allows all boards to communicate reliably and efficiently.

For starters, The communication for most vehicles nowadays use CAN bus protocol, which we will dive more later in this document.

In this document, there will be part where i will say about the main points to think about, And i will also mention the way i will debug the scenario in the end of the case study

2. Main point to think about

2.1. How to identify where to send to

CAN bus doesn't have CS (Chip Select) Pin, or address that you can call like I2C, but it will broadcast to anyone that need the ID tag of the data.

The idea works like BLE service_id, in BLE you have device_id and service_id. But in CAN bus, it only have a bus that every peripherals connects to and take any data with the matching ID that they need.

2.2. Resolver tags

What i will did here to make the CAN bus know where to send to, is to use the ID tag as a sender and reciever tag. I will split the ID tag is split to 11 Bits,

- 3 of the first bits is for the message type / priority.
- 4 of the next bits can be used to identify the destination.
- 4 of the next bits can be used to identify who send it.

It fit perfectly because both of the sender and reciver can be identified.

2.3. Problem

Well, on top of the paper, it seems like a very nice idea. But what if there is more than 16 devices to hook into 1 bus. The implementation that i just made only allows for 16 unique device. Because the limit of 4 bits identification

Also then you miss the sweet spot where CAN is used, which is shared data bus, Because now you don't have the data tag (what data is its sending?)

2.4. Solution

Solution : change to CAN communication protocol allows up to 29 Bit ID (for example : SAE J1939) Now we have more rooms to work on.

Now rather than the split above, you can split it like this

- 3 bits for Priority bits
- 8 bits for Command (data type)
- 8 bits for Reciver ID
- 8 bits for Sender ID

Now what you have, is 256 peripherals that can communicate to each other with 256 commands that can be used to share the data to everyone.

You even have 2 extra bits, to spend on something in the future.

3. How do you define the structure of the message

CAN frame already have 8 Bytes of data that you can use. And this is how i will use it.

- Byte 0 I will use the first byte of the data to tell the structural info, like sequence number or data length (like the frame of the data, if the data doesn't fit the frame snugly, then we know the data is corrupted) .
- Byte 1 - 7 The remaining bytes can be used for the data itself. It can be the sensor readings, motor commands, or alert interrupts.

3.1. How to pack different data (Just examples)

Because different sensors send different types of numbers, we define strict rules for how those 56 data bits are sliced up based on the Command ID (which we defined in the 29-bit CAN ID header). So we didn't just fill the whole 56 bits of data with random values.

3.1.1. For example, Motor Drive Command (Command ID: 0x12)

The Central Computer needs to send speed commands to the left and right wheel motors. We can pack two 16-bit signed integers (shorts) and an 8-bit mode flag into the payload:

- Byte 0: Sequence Counter (0-255) to detect dropped packets
- Bytes 1-2: Left Motor Target Speed (16-bit signed integer)
- Bytes 3-4: Right Motor Target Speed (16-bit signed integer)
- Byte 5: Control Mode (8-bit enum: 0x00 = Idle, 0x01 = Speed Control, 0x02 = Torque Control)
- Bytes 6-7: Unused / Reserved for future growth

3.1.2. Another example, receiving IMU Sensor Readings (Command ID: 0x35)

The IMU needs to send high-precision acceleration data (X, Y, Z axes)

- Byte 0: Sequence Counter, just like before
- Bytes 1-2: X-axis acceleration (16-bit)
- Bytes 3-4: Y-axis acceleration (16-bit)
- Bytes 5-6: Z-axis acceleration (16-bit)
- Byte 7: Unused

4. How do you handle receiving and processing packets at high speed?

For this step, I will actually implement some algorithms and tricks to handle the fast data